

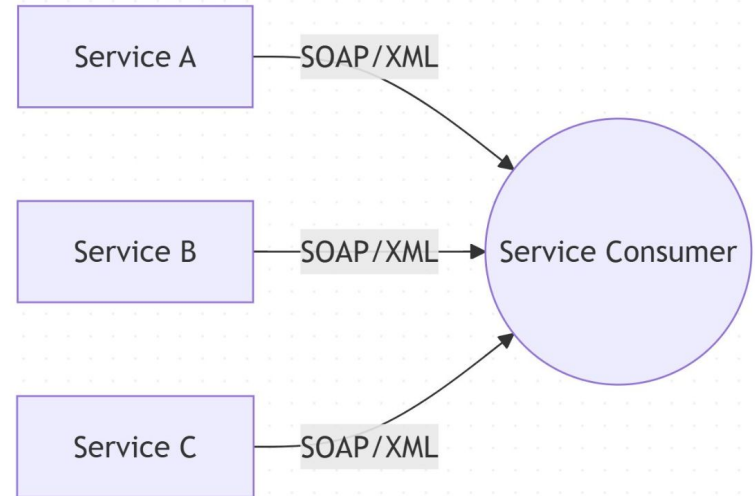


Week 4

- Software Architecture Contd.**
- System Design Example**

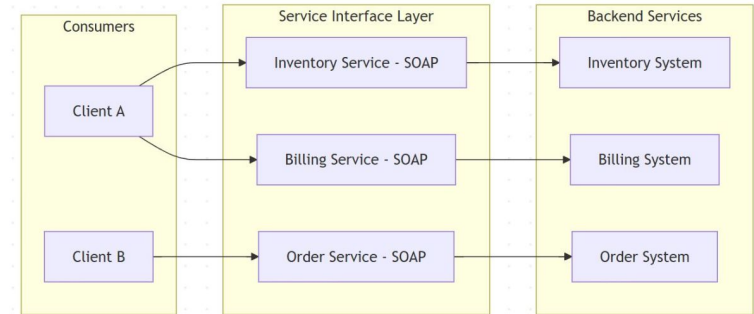
Service Oriented Architecture

- SOA is an architectural style that supports service reusability, loose coupling, and interoperability.
- Services expose well-defined interfaces, typically over a network.
- Services communicate using standardized protocols and formats



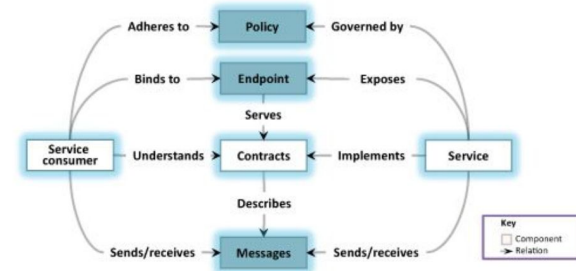
Principles & High level Architecture

- Loose Coupling – Services operate independently.
- Interoperability – Works across platforms.
- Reusability – Services can be reused across apps.
- Discoverability – Services registered and searchable.
- Abstraction – Service hides implementation details.



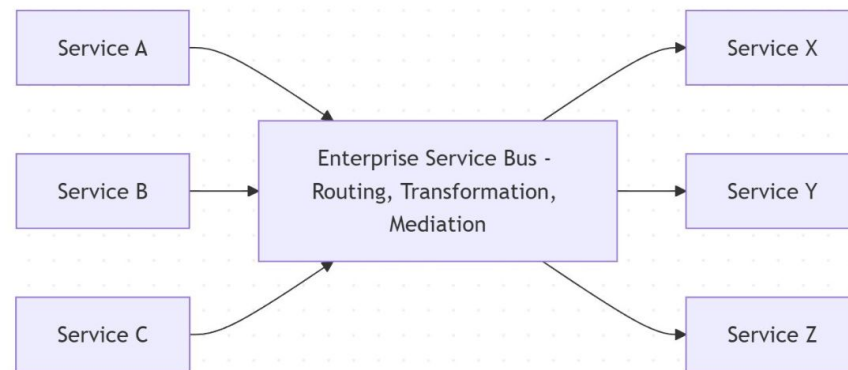
High Level Architecture

- Service consumers call services through a service contract.
- Services are exposed via endpoints.
- Messaging occurs over standard protocols.



Enterprise Service Bus

- Middleware that routes, transforms, and mediates service messages.
- Helps with orchestration, security, monitoring, logging.
- Enables centralized control.



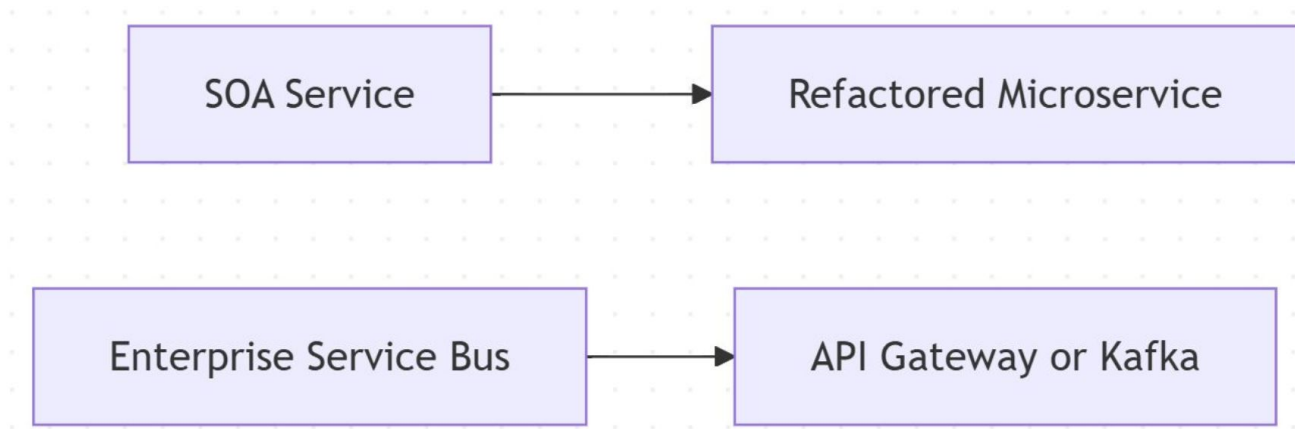


SOA vs Microservice

- Communication Protocols
 - SOA: Uses heavyweight protocols like SOAP and data formats like XML
 - Microservices: Prefer lightweight protocols such as REST/JSON or gRPC
- Service Granularity
 - SOA: Typically coarse-grained services (e.g., BillingService, InventoryService)
 - Microservices: Fine-grained, single-responsibility services (e.g., InvoiceService, PaymentService)
- Integration Style
 - SOA: Often relies on a centralized ESB (Enterprise Service Bus) for routing, transformation, and mediation
 - Microservices: Communicate directly or via lightweight infrastructure (e.g., API Gateway, event brokers)
- Deployment & Scalability
 - SOA: Services are often co-deployed or share infrastructure, making independent scaling harder
 - Microservices: Each service is independently deployable, enabling horizontal scaling per service
- Governance & Control
 - SOA: Emphasizes centralized governance, often with strict contract and schema management
 - Microservices: Support decentralized governance, allowing teams to choose their own tech stacks and release cycles
- Technology Stack
 - SOA: Generally homogeneous, uses enterprise stacks (Java EE, .NET, WebSphere, etc.)
 - Microservices: Promotes polyglot programming, using multiple languages and frameworks across services
- Suitability
 - SOA: Ideal for large enterprises needing integration across legacy systems or B2B workflows
 - Microservices: Better suited for cloud-native, agile teams building modular, rapidly evolving systems

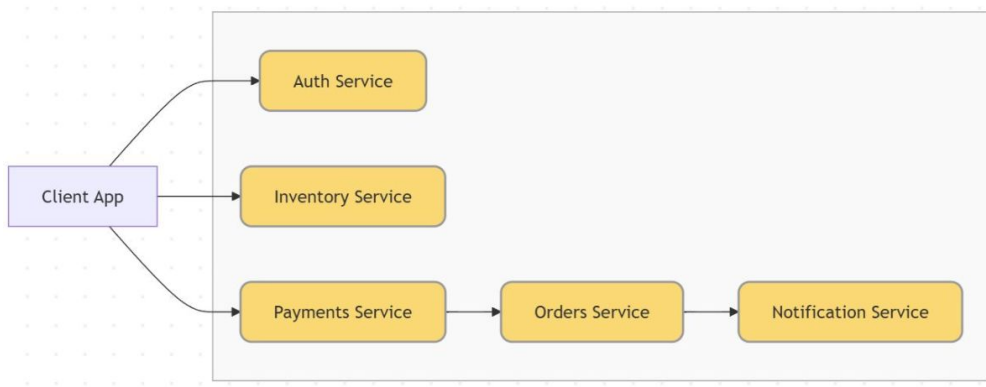
Modernization of SOA to Microservice

- Identify domain boundaries.
- Decompose coarse-grained services.
- Replace ESB with API Gateway / Service Mesh.
- Introduce REST/JSON or gRPC gradually.



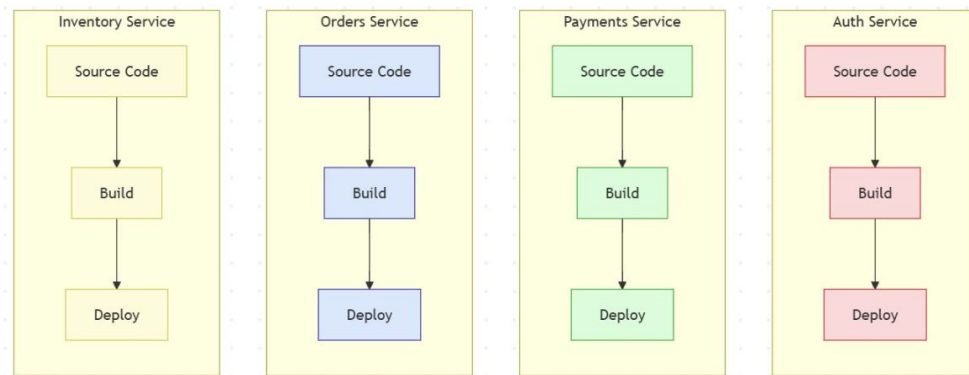
Microservices

- Microservices is an architectural style where applications are composed of small, independent services.
- Each service focuses on a single business capability.
- Services communicate over lightweight protocols (e.g., REST, gRPC, messaging).
- Services are independently deployable and scalable.



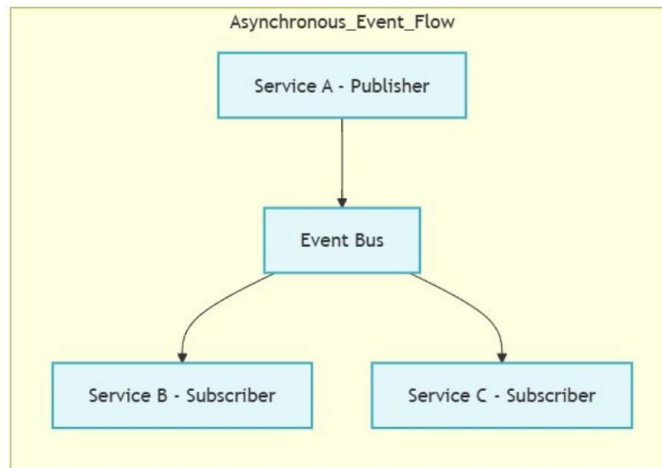
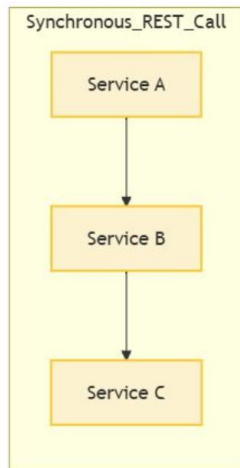
Characteristics

- Single Responsibility: Each service owns one bounded context.
- Autonomous Deployment: Services can be deployed and updated independently.
- Language and Tech Stack Flexibility: Each team can choose their own tools (polyglot).
- Fault Isolation: Failure in one service doesn't bring down the entire system.
- DevOps Friendly: Enables CI/CD and agile delivery.



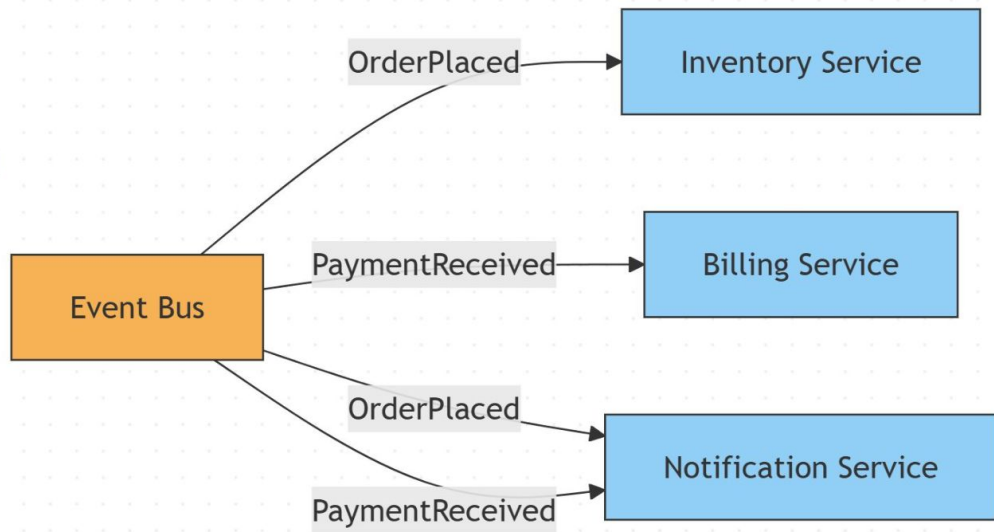
Service Communication Pattern

- Synchronous: REST, gRPC
- Asynchronous: Messaging (Kafka, RabbitMQ, SQS)
- Services interact via APIs, contracts, and events.
- Must handle network failures and timeouts gracefully.



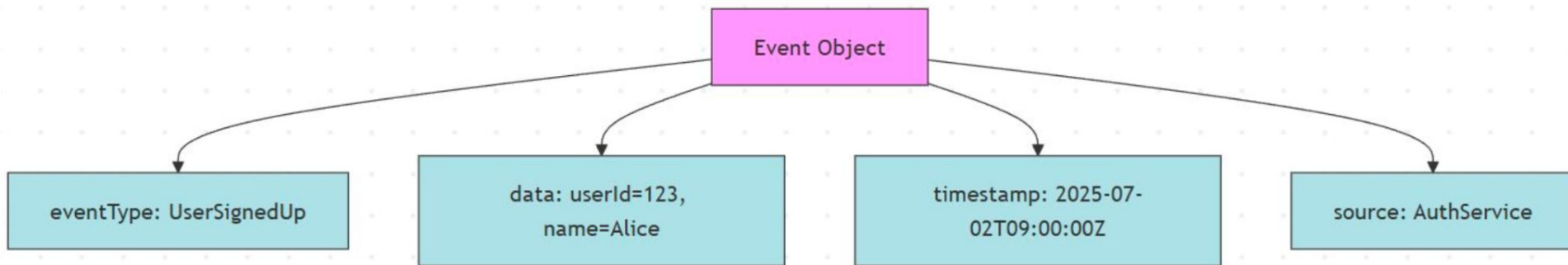
Event Driven Architecture

- Software design paradigm centered around events
- Promotes loose coupling and high scalability
- Enables asynchronous communication between components



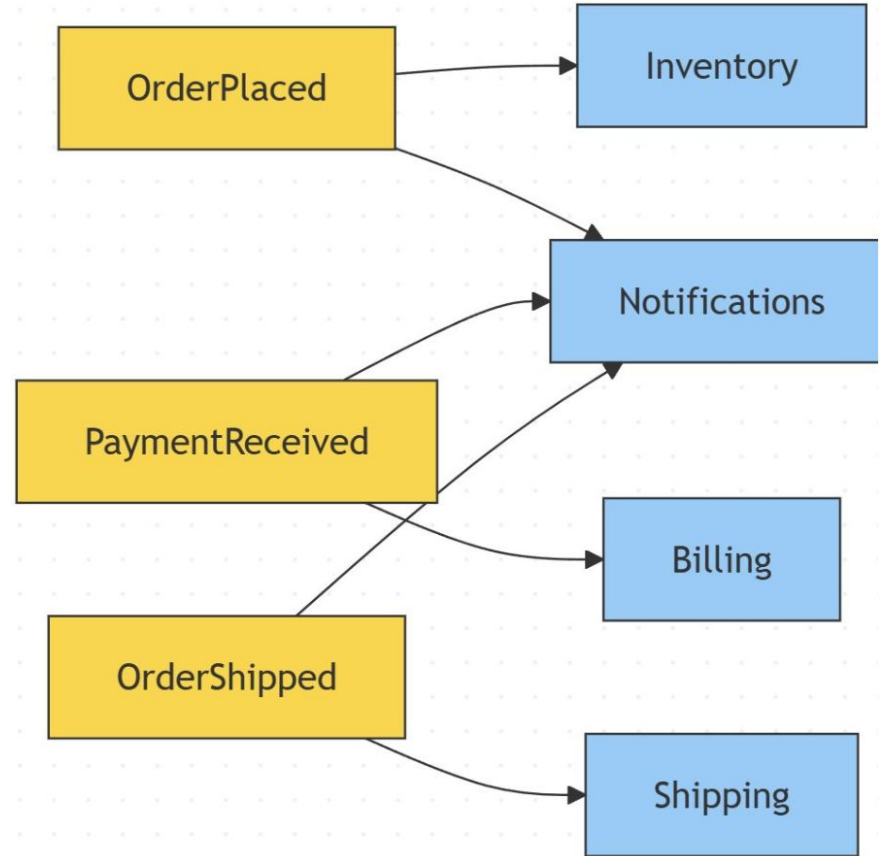
What's an Event

- Represents a significant state change (e.g., "UserSignedUp")
- Immutable and descriptive
- Contains metadata (timestamp, ID, source)



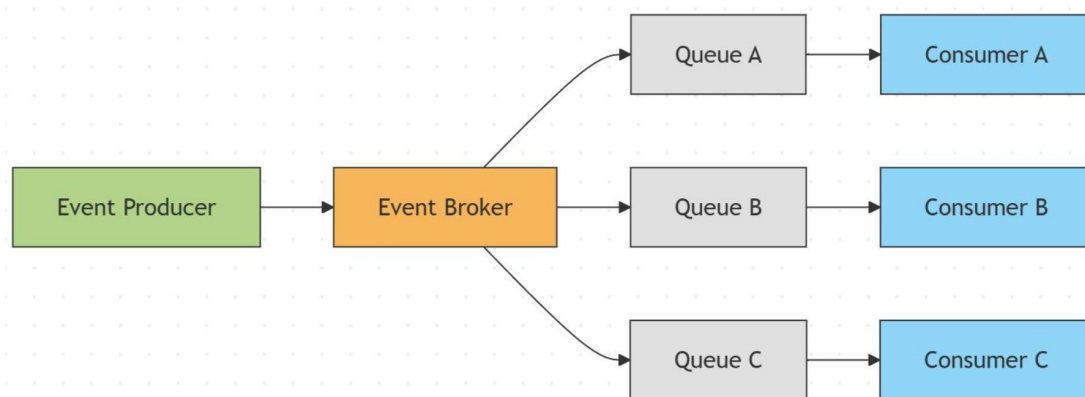
Event As First Class Citizen

- Events drive logic, workflows, and data flows
- Stored, versioned, and replayable
- Central to system design, not just triggers



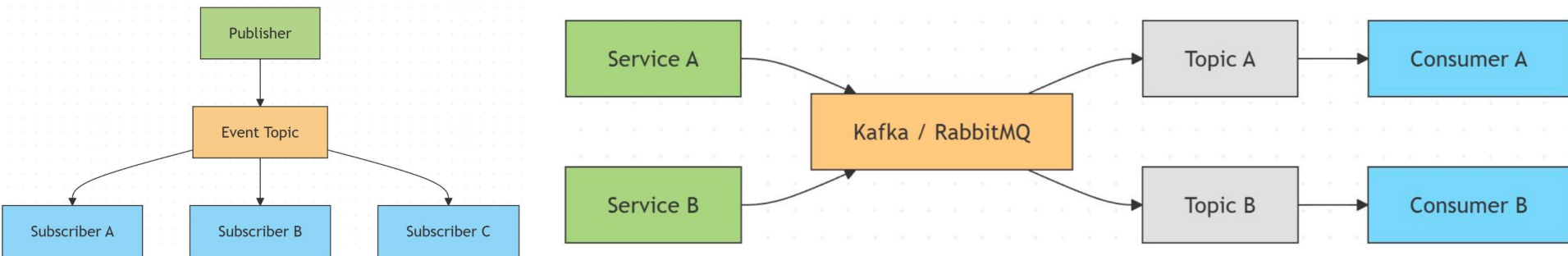
Asynchronous flow

- Producers emit events without waiting
- Consumers handle events at their own pace
- Improves scalability and resilience



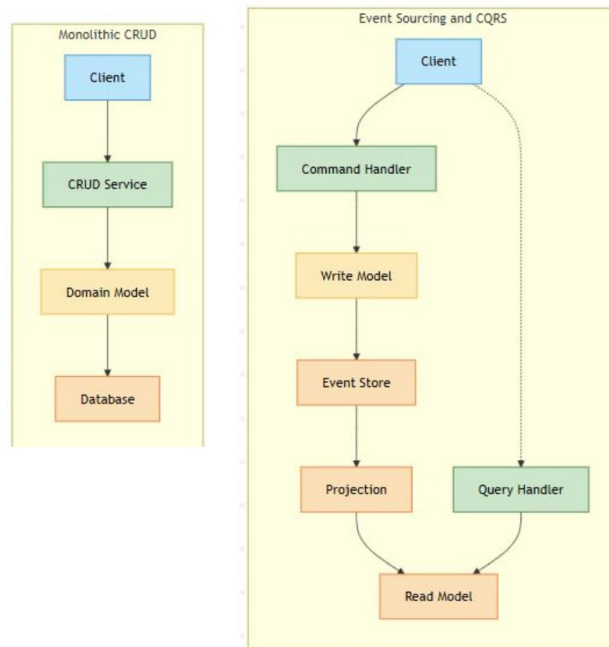
Publisher & Event Broker

- Events published to a topic/channel
- Multiple subscribers receive relevant events
- Promotes decoupling between senders and receivers
- Middleware that routes events (e.g., Kafka, RabbitMQ)
- Handles delivery, retries, and message durability
- Often decouples services via topics or queues



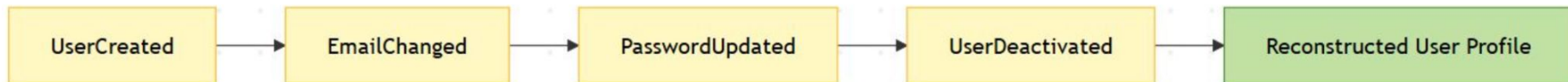
Event Sourcing + CQRS Architecture

- Modern systems demand scalability, auditability, and performance
- Traditional CRUD models can mix responsibilities, reducing flexibility
- Event Sourcing captures every state change as an event
- CQRS splits command (write) and query (read) responsibilities for clarity and optimization



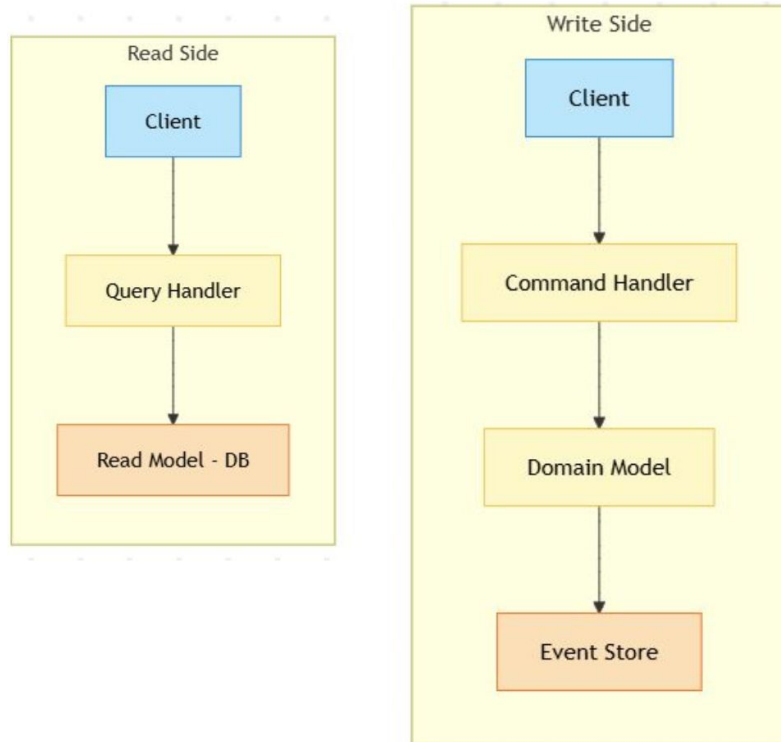
What's Event Sourcing

- State is rebuilt by replaying a sequence of past events
- Events are immutable and append-only
- Event log is the source of truth, not a database record
- Enables full audit trails and historical debugging



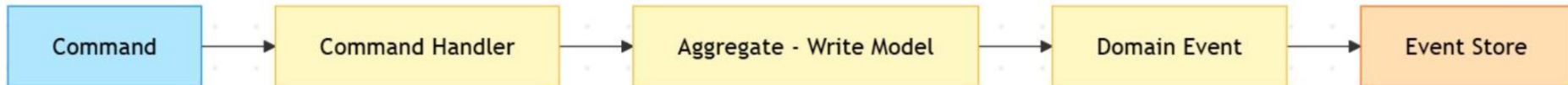
What's CQRS

- Commands = write operations (intent to change state)
- Queries = read operations (no side effects)
- Separation allows independent optimization
- Encourages clear boundaries and scalability



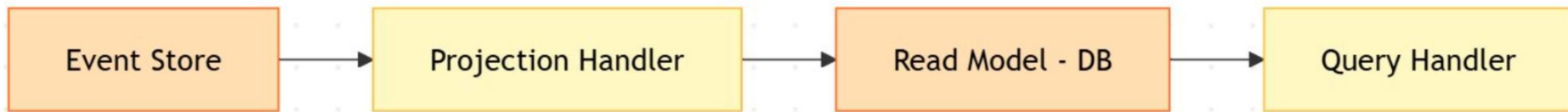
CQRS - Write Model

- Handles domain logic and business rules
- Emits events upon state changes
- Events are persisted in the event store
- Commands are validated and processed synchronously



CQRS - Read Model

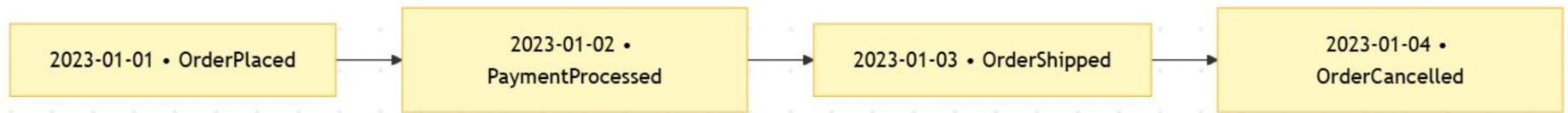
- Built from projected events for fast querying
- Can use optimized schemas (denormalized, indexed)
- Can have multiple views for different use cases
- Eventual consistency with write model





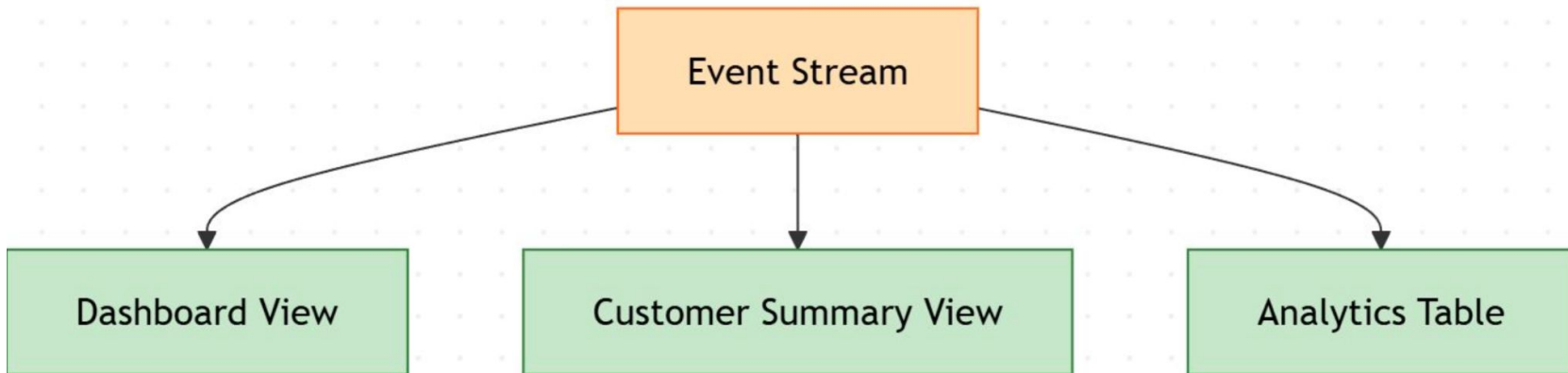
Event store - The Heart of the System

- Central log of all domain events
- Append-only, versioned, and timestamped
- Enables rebuilding state and projections
- Supports auditing, replay, and debugging



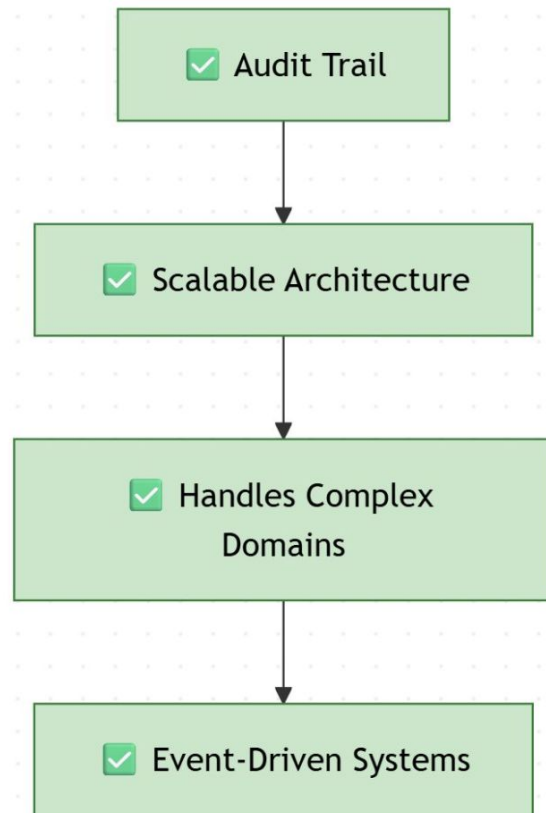
Projections - Building Read Models

- Transform events into queryable views
- Can be rebuilt anytime by replaying events
- Multiple projections can be derived from the same event stream
- Supports polyglot persistence and custom schemas



When to use Event Sourcing + CQRS

- Domains with complex logic and audit needs
- Systems with high read/write asymmetry
- Use cases demanding traceability or eventual consistency
- Event-driven microservices architectures





Modularity

- Coupling - How well modules work together
- Cohesion - How well module meets a singly designed goal

Goals of Modularity

- Abstraction
- Decomposability
- Composability
- Module Understability
- Continuity
- Protectability



System Design Example - URL Shortener System

Functional requirements describe what the system is supposed to do. Two questions:
Which problem does the system solve for the user?

- Focussed on the general problem user want to solve
- Purpose define function and non functional requirement

For ex. In the case of Todo, users want to become more productive and not forget important everyday

- Features - Simple list to show the open items and be able to sort them by due date
- Non Functional requirement - Low latency

Features are essential to solve these problems?

- Core Features: Bare Minimum of what's needed to solve the problem
 - For ex. Todo , create, modify and complete todo items
- Support Features - Help users to solve problem in a convenient way
 - Todo list items, due date



Initial System Analysis -

Read or Write heavy

- Streaming applications are a typical read heavy system, as millions of users watch or listen to the available content while just a fraction of users uploads new content.
- Web analytics tools. On the other hand, something like Google Analytics are a good example for write heavy systems.

Impact on Architecture

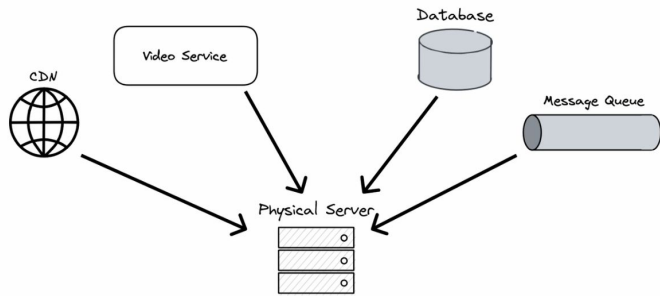
- Scale databases
- Redundancy of services
- Effectiveness of caching

Priority System Architecture

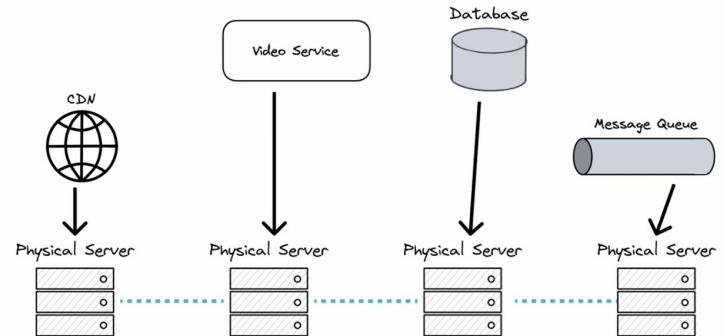
Read Heavy - You would rather want to have a read heavy system that fails to write, than one that fails to read.

Cheat Code - Most of the systems you will come across are read heavy.

Single or Distributed Architecture



Monolithic



Distributed



Non Functional Requirements

Availability describes how long a system is up and running per year.

- Five nines, 99.999% of the time available
- Down for a maximum of 5.15 minutes a year

Consistency

Data Consistency - The data is consistent if it appears to be the same in all corresponding nodes at the same time regardless.

Data Consistency = Linearizability



Scalability - Scalability describes the ability of a system to handle a fast growing user base, and temporary load peaks seamlessly.

Relevant to every system design.

Two ways to scale a system - Vertical & Horizontal

- Vertical - Relying on a single bulky server
- Horizontal - cluster of regular servers that are all hosting instances of system components in parallel.



Latency: Latency by itself is the amount of time it takes for a single message to be delivered.

The concept can be applied to any aspect of a system where data is being requested or transferred.

Latency = Network + System latency

Impact of Design Choices:

Low latency is a priority:

- Database and data model directly influence how long some queries take. Thereby, databases can be a substantial source of latency.
- Caching can also be effective for lower latencies.



Compatibility describes the ability of a system to operate seamlessly with other software, hardware and systems.

Important for systems that target a wide and diverse user base.

For example, YouTube functioning independently of given

- Network bandwidth
- Web browser
- Client Device



Security - Mostly out of scope

- Very complex (breadth & Depth)
- Only make it NFR if you are very confident

Instead corporate standard security measures in your design

- TLS encrypted network traffic
- API Keys for rate limiting

Scalability - Expected scale is essential to create suitable architecture.

Total scale of the expected user base is essential to come up with a suitable system architecture.



Daily Active User (DAU)

- Key metrics to define the scale
- Active users per month (MAU) is common too

Peak Active User

- Triggered by global events
- Lead to a x times, average DAU
- Good system design can handle peaks

Interactions per User

- Find out one kind of interactions (read or write)
- Use read-write ratio to get holistic view
- Focus only on core features



Request Size

- Size of the payload of the request
- Important to define the scale besides DAU and active user

Replication Factor (*Standard factor is 3*)

- Data is replicated to multiple node
- Increases read scalability
- Creates redundancy
- Takes up more storage capacity



Encoding

Goal : Short yet unique key , guaranteed to not collide with existing ones is not trivial.

Parameters to influence:

- Length of the key
- Range of allowed characters

Approach:

- Create keys with short & fixed length
- Allows a wide range of Characters per digit



Allowable Encoding Schemas

Base 36

- [a-z]
- [0-9]

Base 62

- [a-z]
- [A-Z]
- [0-9]

Base 64

- [a-z]
- [A-Z]
- [0-9]
- +, /



Option 1 - MD 5 Hash Function

MD 5 Hash function

- Quickly and low computation power required
- Hash URL into fixed 128 bit string

Issues:

- MD5 algorithm is flawed, uniqueness not guaranteed.
- Increases the risk of collision
- Preventing collision by additional logic is complex.



Option 2

Encoded Counter

- Increases every time a new short link is created
- Create uniques and short keys (for low count value)
- Very scalable and no database required

Issues

- No fixed length
- Slowly increasing length

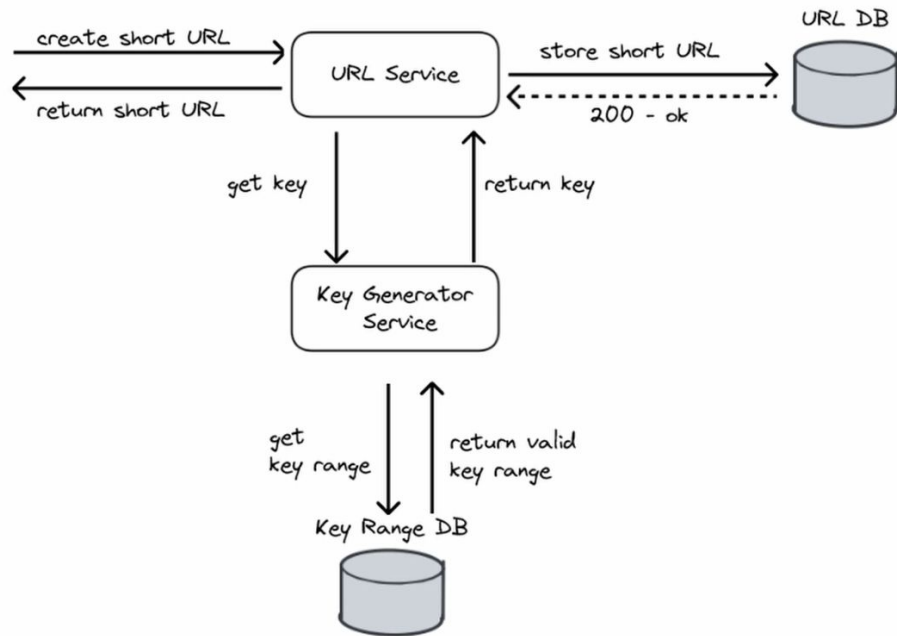
Option 3 - Key Range

- Generate 6 digit key before hand
- Store key in database
- Truly unique keys, no collision

Issues

- Storing all 6 digit means 68 billion entries
- Storage resource intensive

Architecture





Thank You